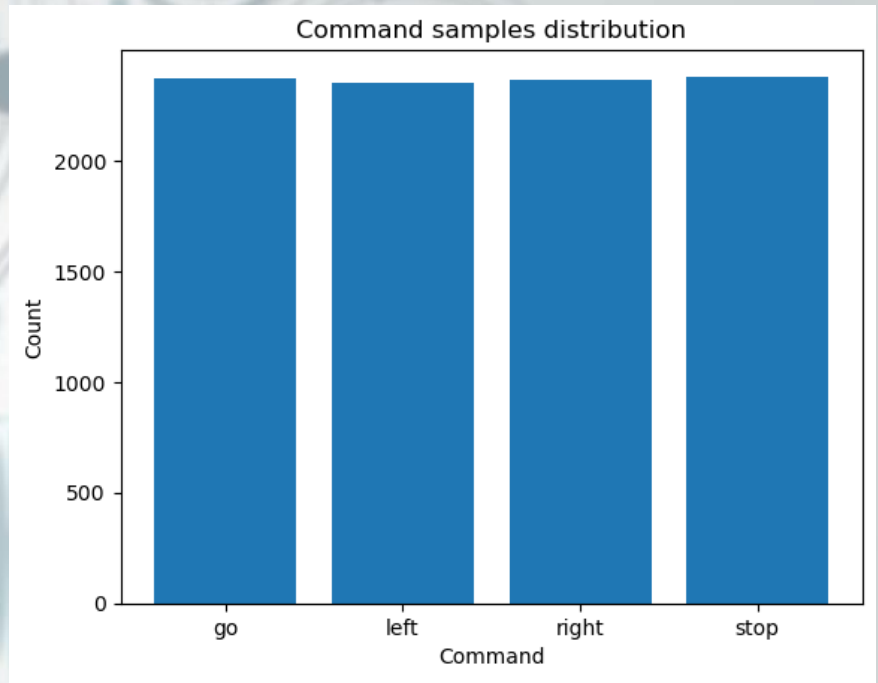


Klasifikácia hlasových povelov

Martin Berki | Michal Lúčny

Dataset

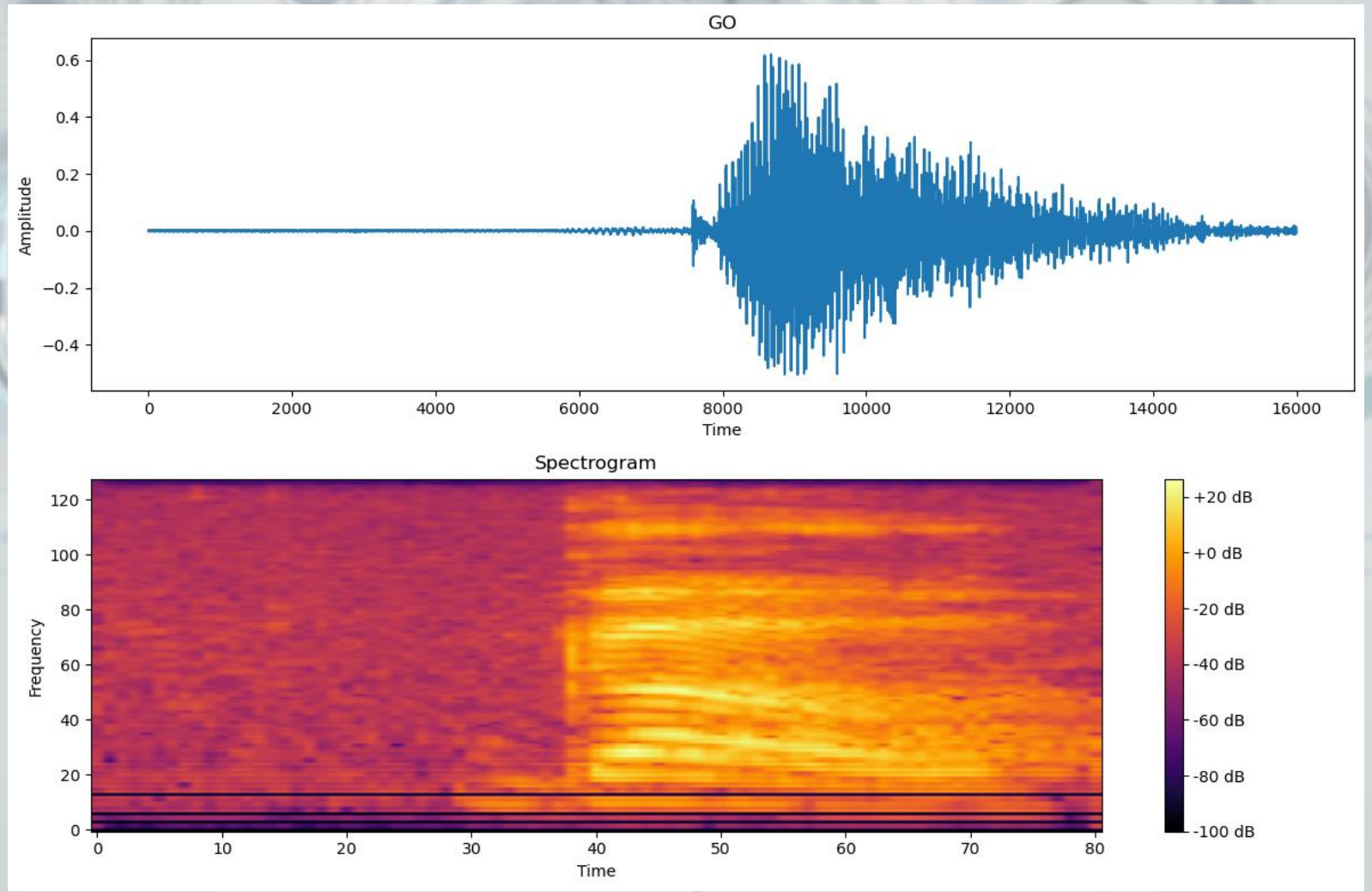
- 4 hlasové povely
  - GO, LEFT, RIGHT, STOP
- Približne 10K vzoriek
- .WAV Formát
- 1 vzorka = 1 sekunda pri FS 16kHz
- Rozdelenie dát
  - 0.8 : 0.1 : 0.1
- Šumové dáta variabilnej dĺžky
  - Biely šum, Ružový šum, Mačka, Umývačka riadu, Bicykel



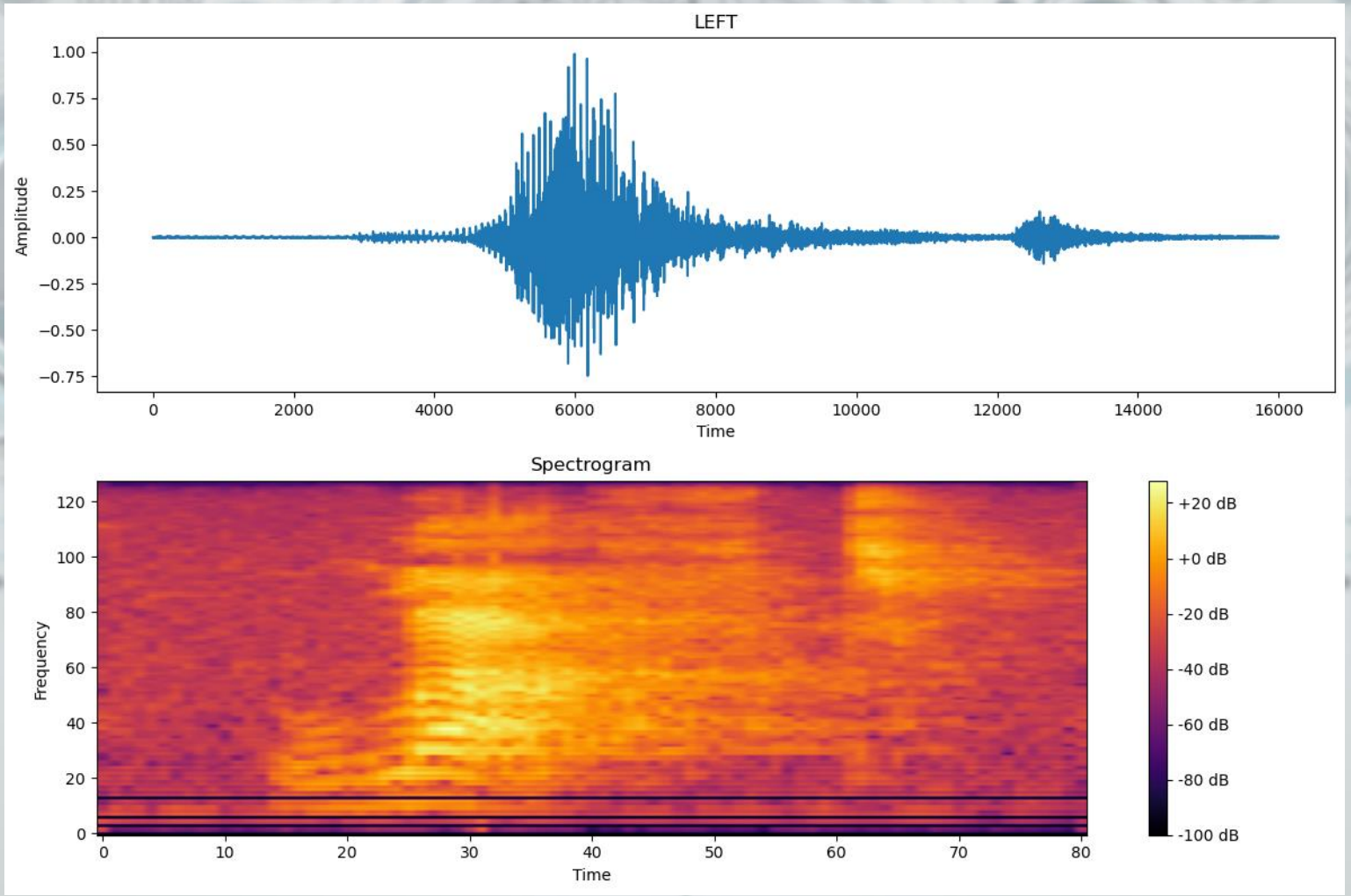
Predspracovanie dát

- Trénovanie v dvoch režimoch
  - RAW signál
  - Spektrogram
- Downsampling
- Zero batch-padding
- Náhodné pridávanie šumu s náhodnou mierou zosilnenia (max. 0.3)

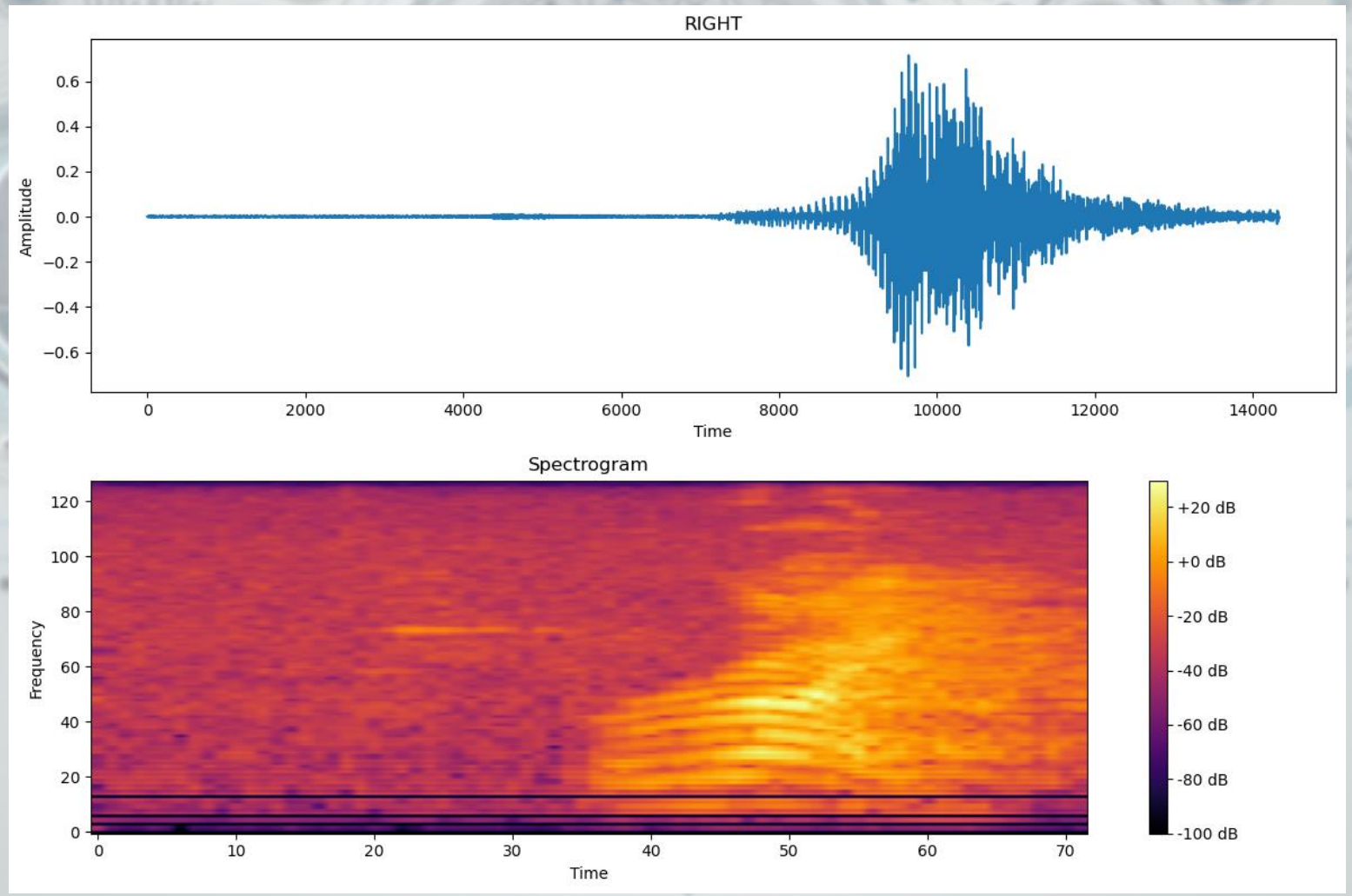
GO


GO
Amplitude
0.6
0.4
0.2
0.0
−0.2
−0.4
0
2000
4000
6000
8000
10000
12000
14000
16000
Time
Spectrogram
Frequency
120
100
80
60
40
20
0
0
10
20
30
40
50
60
70
80
Time
+20 dB
+0 dB
-20 dB
-40 dB
-60 dB
-80 dB
-100 dB

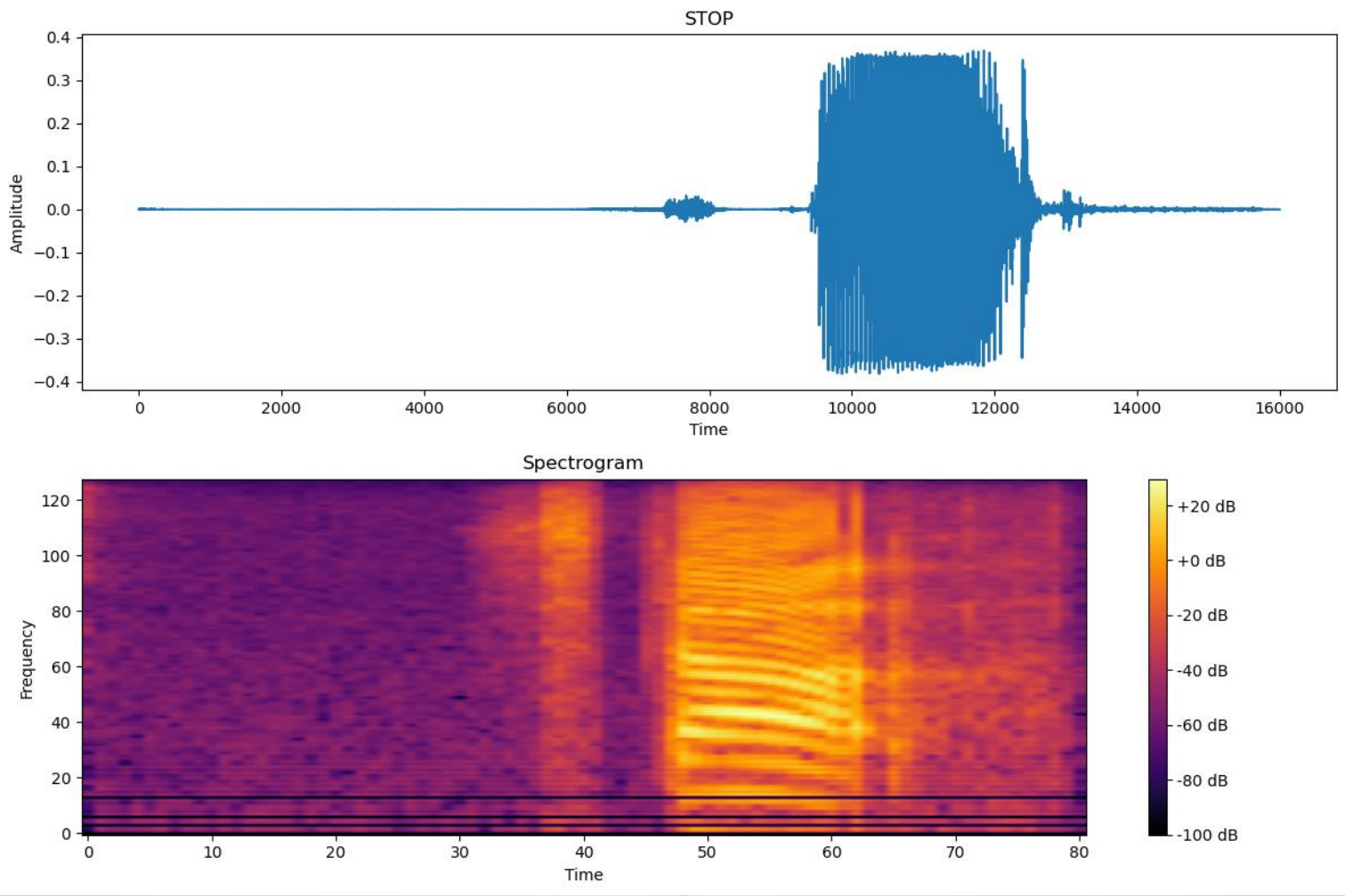
LEFT


LEFT
Amplitude
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
0
2000
4000
6000
8000
10000
12000
14000
16000
Time
Spectrogram
Frequency
120
100
80
60
40
20
0
0
10
20
30
40
50
60
70
80
Time
+20 dB
+0 dB
-20 dB
-40 dB
-60 dB
-80 dB
-100 dB

RIGHT

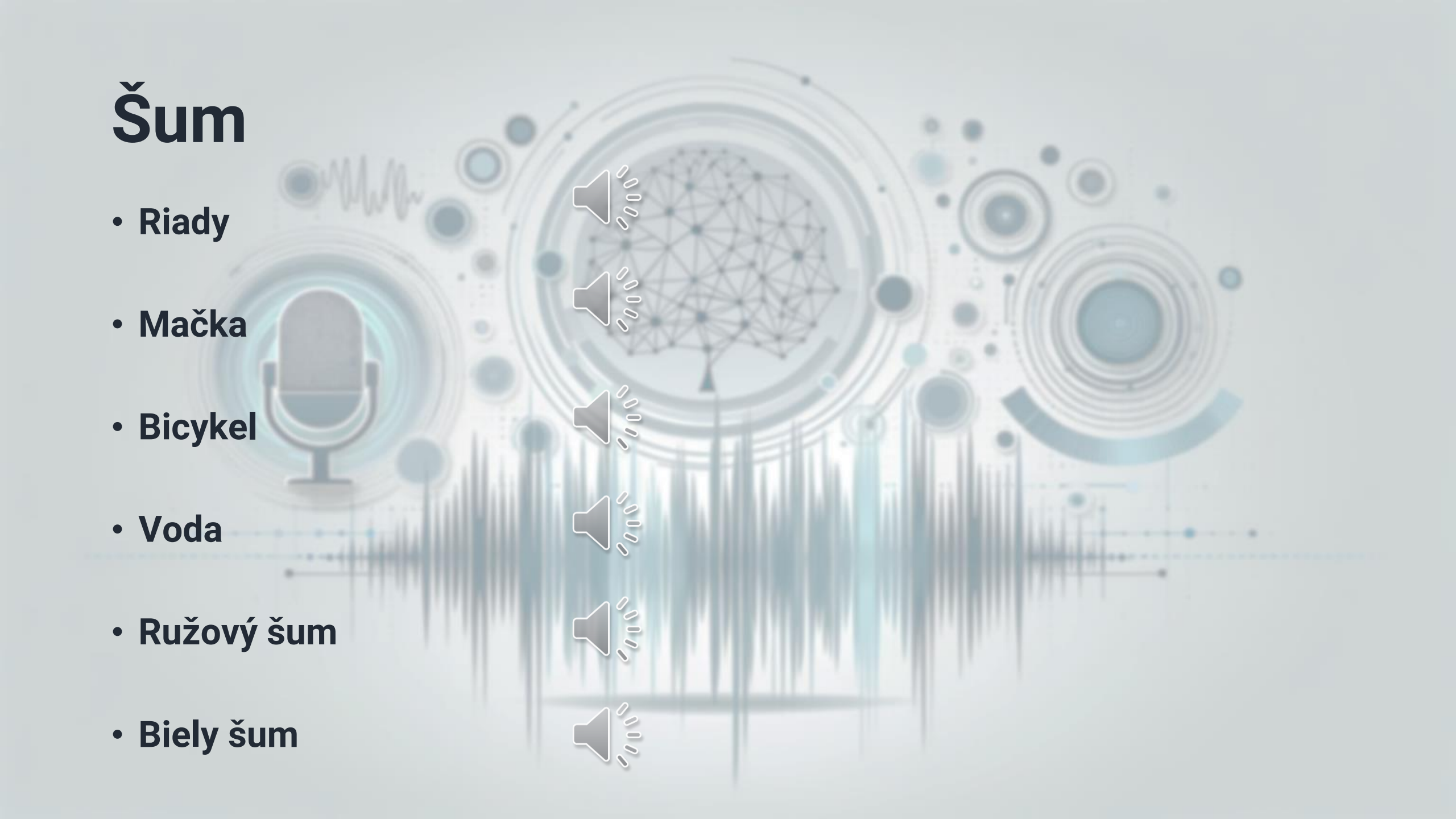

RIGHT
Amplitude
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6
0
2000
4000
6000
8000
10000
12000
14000
Time
Spectrogram
Frequency
120
100
80
60
40
20
0
0
10
20
30
40
50
60
70
Time
+20 dB
+0 dB
-20 dB
-40 dB
-60 dB
-80 dB
-100 dB

STOP


STOP
Amplitude
Time
Spectrogram
Frequency
Time
+20 dB
+0 dB
-20 dB
-40 dB
-60 dB
-80 dB
-100 dB

Šum

- Riady
- Mačka
- Bicykel
- Voda
- Ružový šum
- Biely šum



Modely

- **CNN – LSTM**
- **CNN (M5)**

- **Trénovanie na RAW signály**
- **Trénovanie na spektrograme**

CNN - LSTM

- Vlastná štruktúra
  - Konvolučná časť – Reprezentácia príznakov
  - Rekurentná časť – Sekvenčná reprezentácia
- Parametre
  - Trénovanie na RAW dátach : 703 364 parametrov
  - Trénovanie na spektrograme: 191 172 parametrov
- Štruktúra
  - RAW : 1 vstup , 64 LSTM modulov, 2 rekurentné vrstvy, 4 výstupy
  - SPG: 128 vstupov, -//-
- Sekvencie
  - RAW: (1,8000,1)
  - SPG: (1, 81, 128) (obe treba transponovať kvôli konvolúcii)

```python
class CNNLSTMModel(nn.Module):
    def __init__(
        self,
        input_size,
        hidden_size,
        num_layers,
        output_size,
        use_specgram=False,
        n_channels=64,
    ):
        super(CNNLSTMModel, self).__init__()
        self.hidden_size = hidden_size
        self.num_layers = num_layers
        self.use_specgram = use_specgram

        # Define the convolutional layer
        self.conv1 = nn.Conv1d(
            in_channels=input_size,
            out_channels=n_channels,
            kernel_size=3 if self.use_specgram else n_channels,
            stride=1 if self.use_specgram else 16,
        )
        self.bn1 = nn.BatchNorm1d(num_features=n_channels)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool1d(4)

        if not self.use_specgram:
            self.conv2 = nn.Conv1d(n_channels, n_channels, kernel_size=3)
            self.bn2 = nn.BatchNorm1d(n_channels)
            self.relu2 = nn.ReLU()
            self.pool2 = nn.MaxPool1d(4)

        # Define the LSTM layer
        self.lstm = nn.LSTM(
            n_channels,
            hidden_size,
            num_layers,
            batch_first=True,
            bidirectional=True,
            dropout=0.25,
        )

        # Define the fully connected layer
        self.fc = nn.Linear(hidden_size * 2, output_size)

        self.sigmoid = nn.Sigmoid()
```

```python
def forward(self, x):
    x = x.permute(0, 2, 1)

    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu1(x)
    x = self.pool1(x)

    if not self.use_specgram:
        x = self.conv2(x)
        x = self.bn2(x)
        x = self.relu2(x)
        x = self.pool2(x)

    x = x.permute(0, 2, 1)

    # Initialize LSTM hidden state
    h0 = torch.zeros(self.num_layers * 2, x.size(0), self.hidden_size).to(x.device)
    c0 = torch.zeros(self.num_layers * 2, x.size(0), self.hidden_size).to(x.device)

    # Forward pass through LSTM
    out, (h0, c0) = self.lstm(x, (h0, c0))

    # Get the last time step's output
    out = self.fc(out[:, -1, :])

    out = self.sigmoid(out)

    return out
```

CNN (M5)

- Výhradne konvolučná štruktúra navrhnutá na klasifikáciu RAW signálov [https://arxiv.org/pdf/1610.00087.pdf]
- Parametre
  - RAW: 25 412
  - SPG: 16 068

```python
class M5(nn.Module):
    def __init__(self, n_output=4, stride=16, n_channel=32, use_specgram=False):
        super().__init__()
        self.stride = stride
        self.use_specgram = use_specgram

        self.conv1 = nn.Conv1d(
            in_channels=128 if self.use_specgram else 1,
            out_channels=n_channel,
            kernel_size=3 if self.use_specgram else 80,
            stride=1 if self.use_specgram else self.stride,
        )
        self.bn1 = nn.BatchNorm1d(n_channel)
        self.pool1 = nn.MaxPool1d(4)
        self.conv2 = nn.Conv1d(n_channel, n_channel, kernel_size=3)
        self.bn2 = nn.BatchNorm1d(n_channel)
        self.pool2 = nn.MaxPool1d(4)

        if not self.use_specgram:
            self.conv3 = nn.Conv1d(n_channel, 2 * n_channel, kernel_size=3)
            self.bn3 = nn.BatchNorm1d(2 * n_channel)
            self.pool3 = nn.MaxPool1d(4)
            self.conv4 = nn.Conv1d(2 * n_channel, 2 * n_channel, kernel_size=3)
            self.bn4 = nn.BatchNorm1d(2 * n_channel)
            self.pool4 = nn.MaxPool1d(4)

        self.fc1 = nn.Linear((4 if self.use_specgram else 6) * n_channel, n_output)
        self.sigmoid = nn.Sigmoid()
```

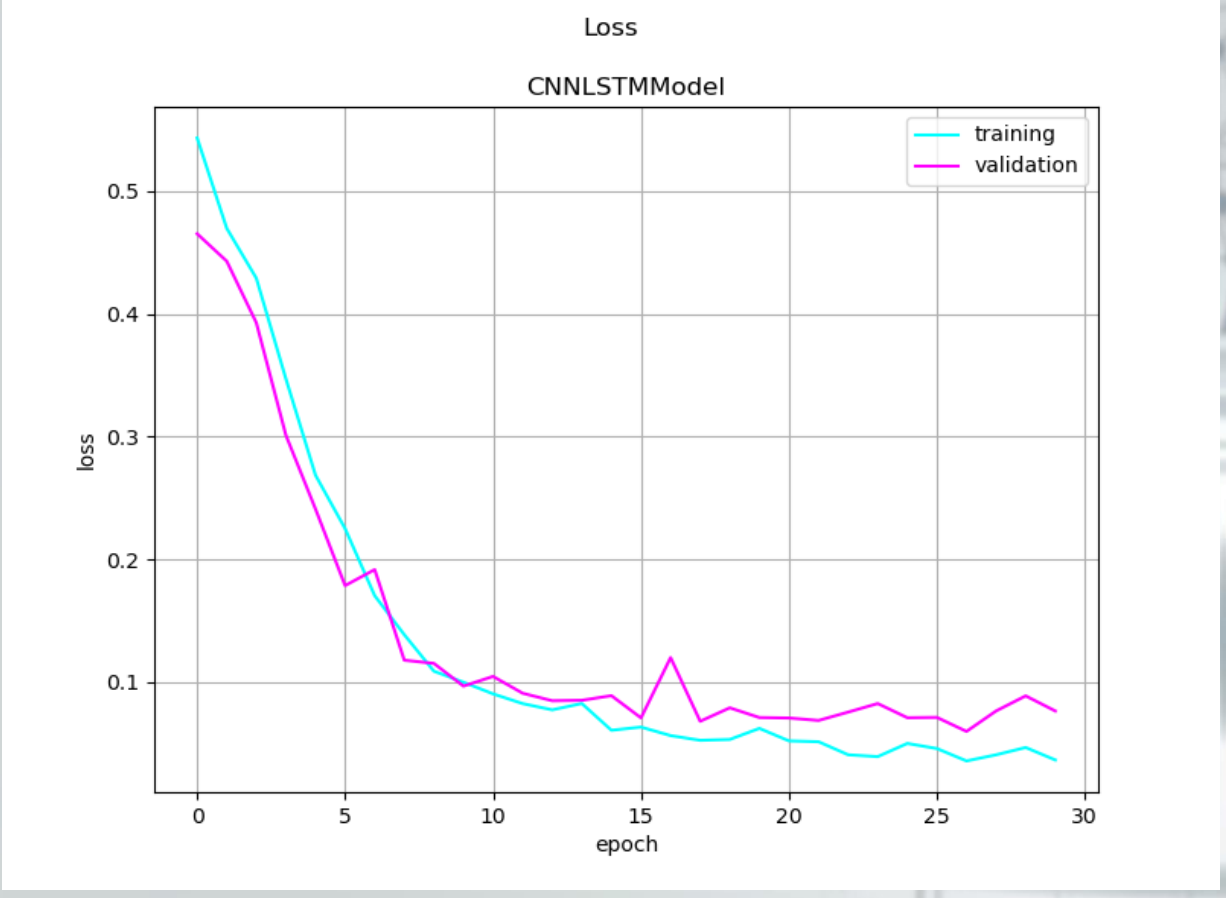
```python
def forward(self, x):
    x = x.permute(0, 2, 1)
    x = self.conv1(x)
    x = F.relu(self.bn1(x))
    x = self.pool1(x)
    x = self.conv2(x)
    x = F.relu(self.bn2(x))
    x = self.pool2(x)

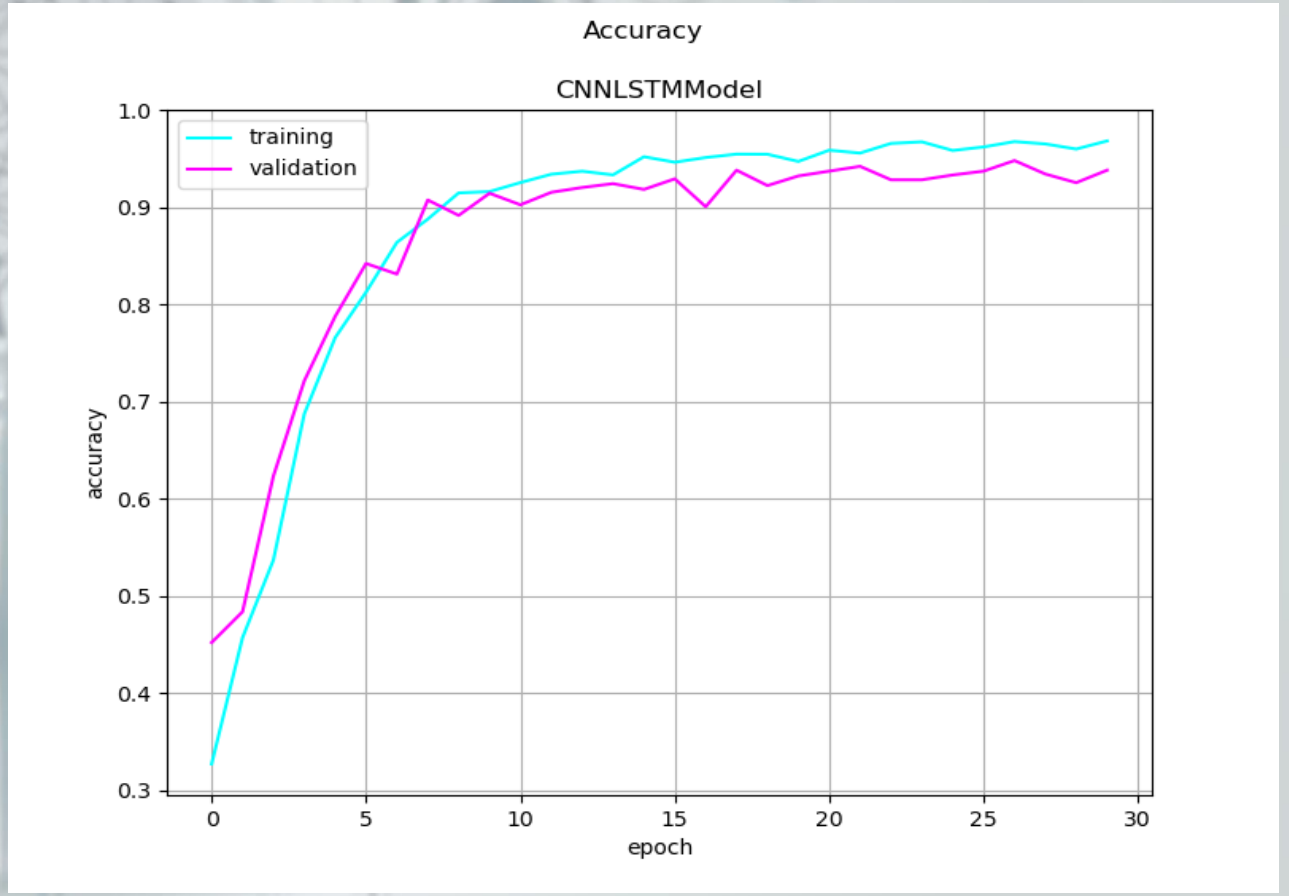
    if not self.use_specgram:
        x = self.conv3(x)
        x = F.relu(self.bn3(x))
        x = self.pool3(x)
        x = self.conv4(x)
        x = F.relu(self.bn4(x))
        x = self.pool4(x)

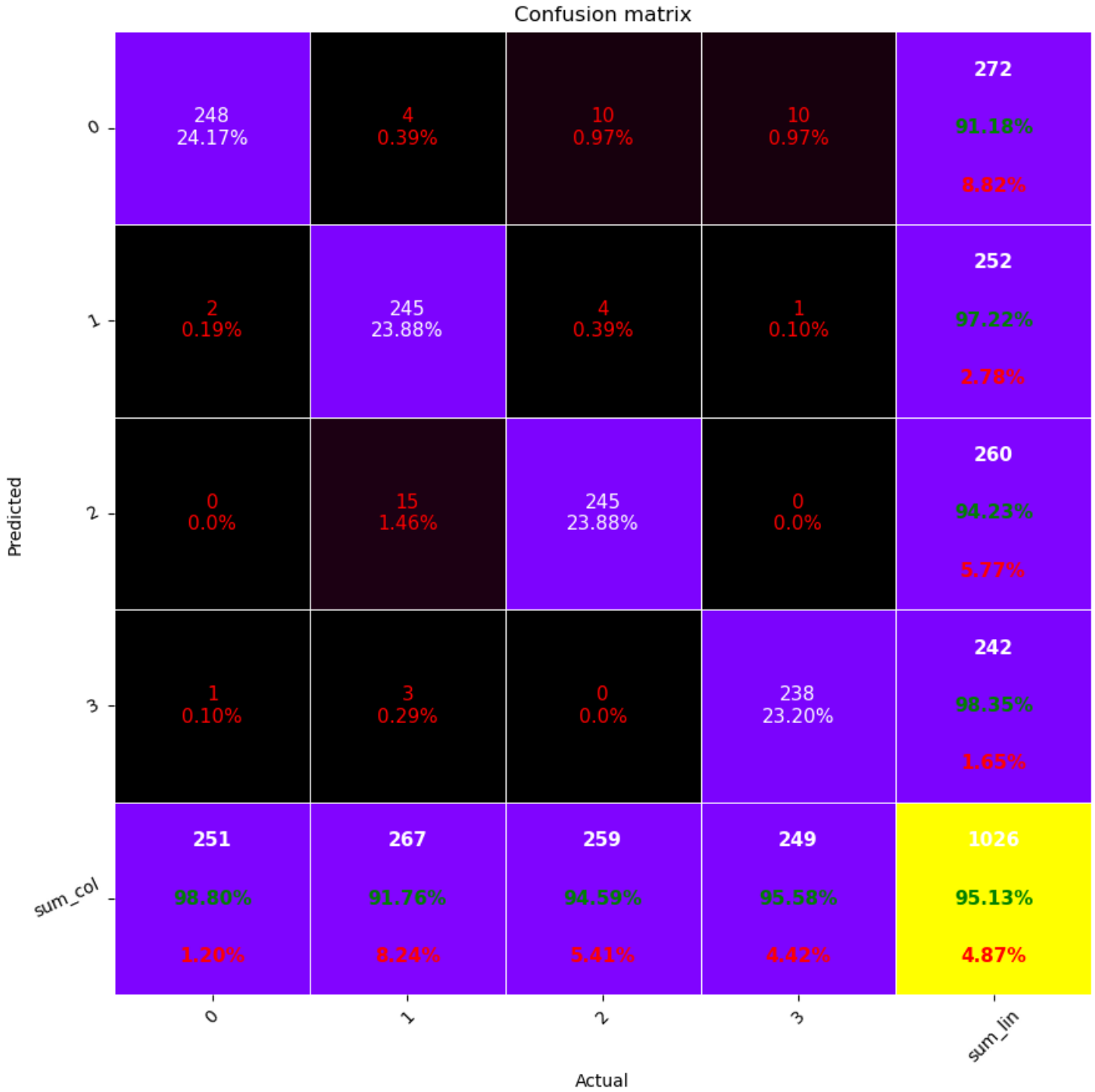
    x = x.view(x.size(0), -1)
    x = self.fc1(x)
    out = self.sigmoid(x)

    return out
```

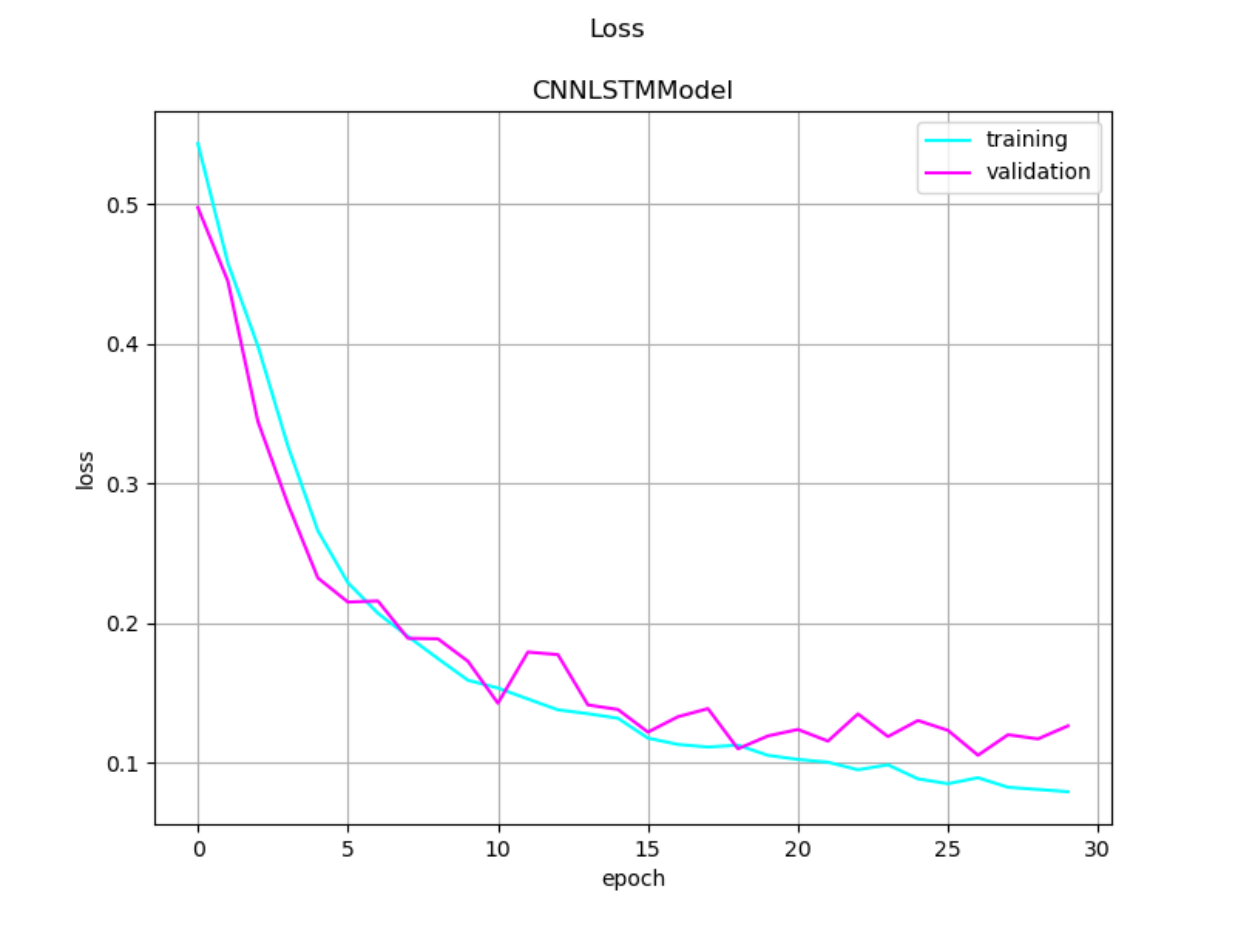
Trénovanie CNN – LSTM na RAW dátach

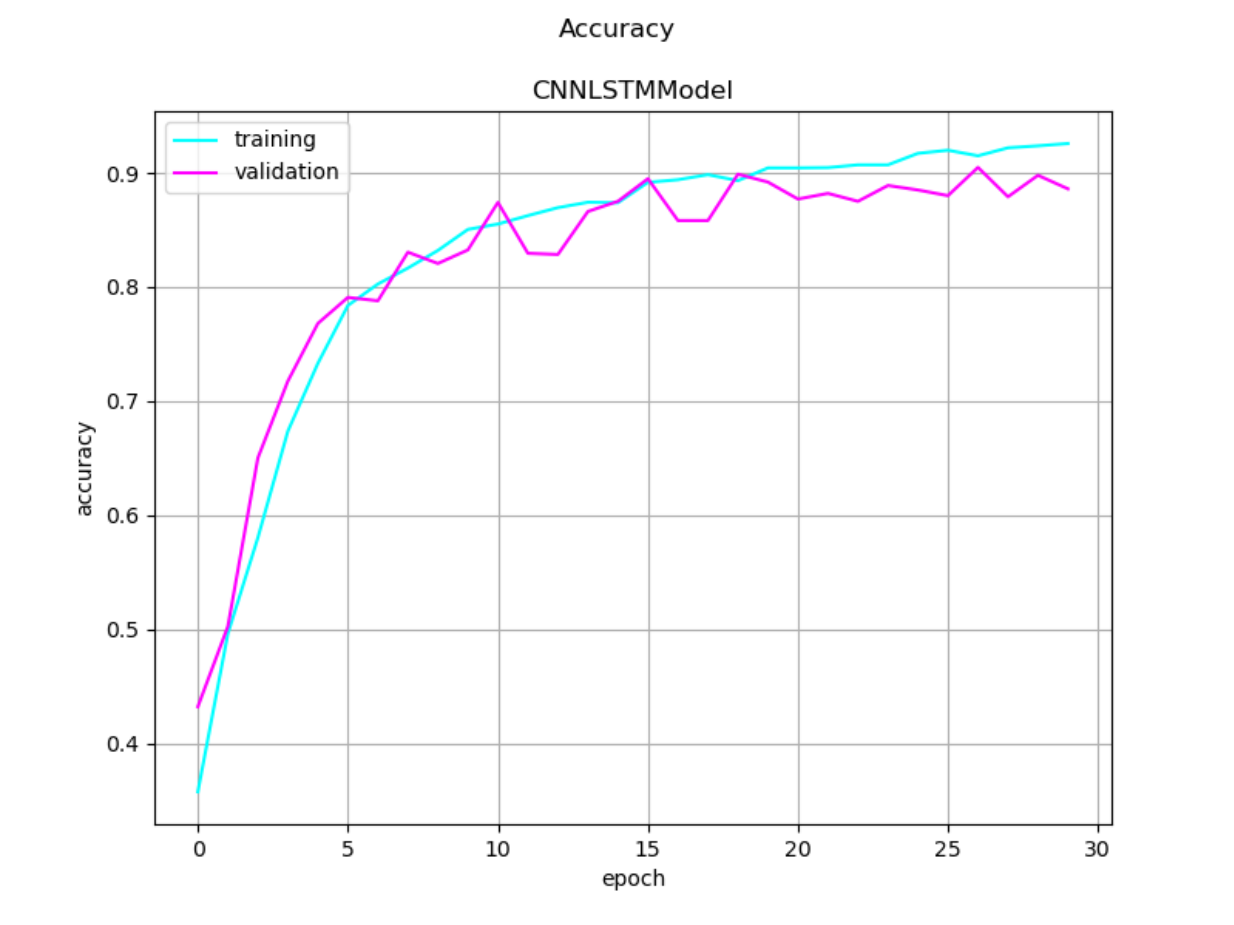


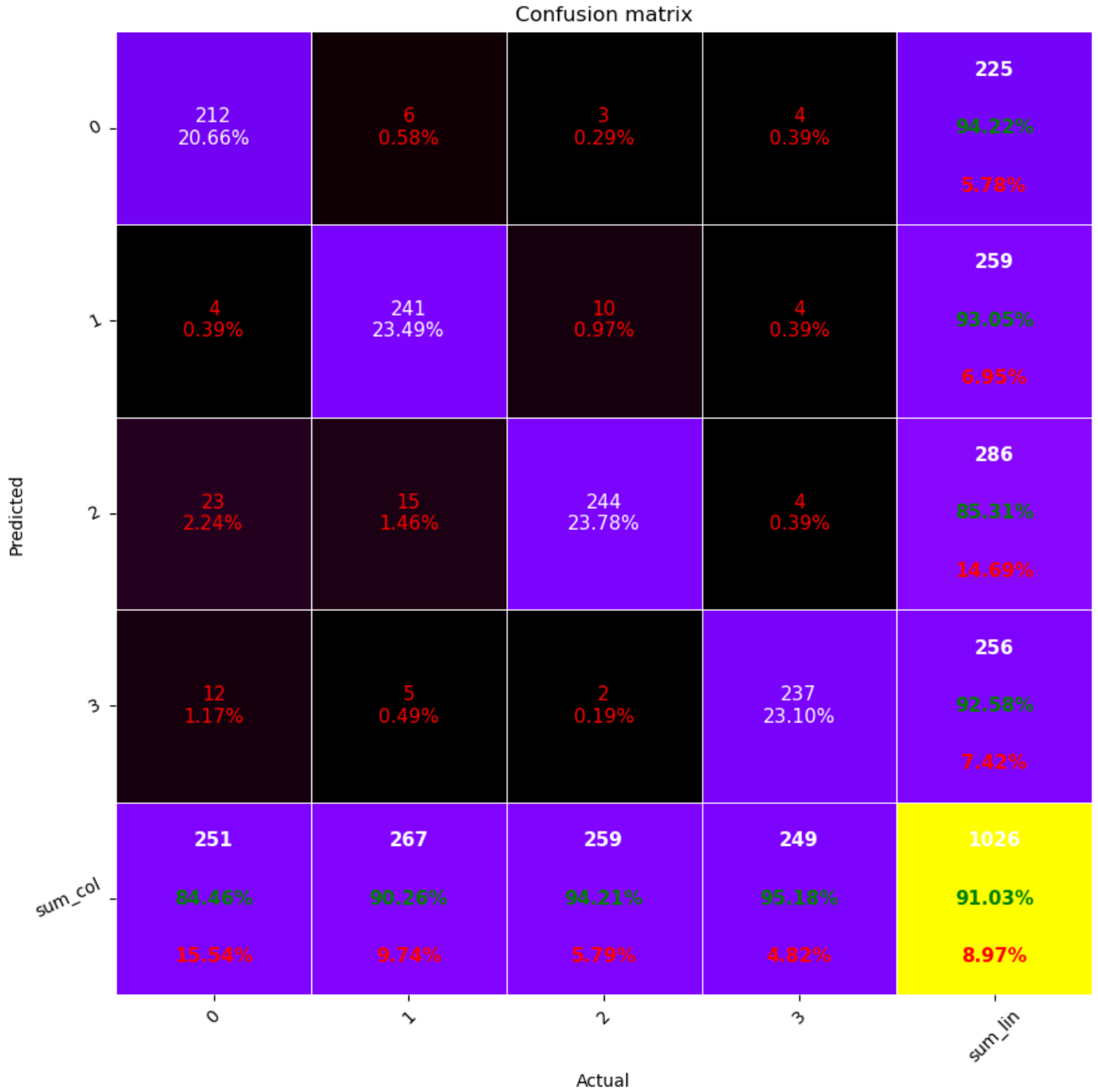



Confusion matrix
Predicted
Actual
0
1
2
3
sum_col
sum_lin
248
24.17%
4
0.39%
10
0.97%
10
0.97%
272
91.18%
8.82%
2
0.19%
245
23.88%
4
0.39%
1
0.10%
252
97.22%
2.78%
0
0.0%
15
1.46%
245
23.88%
0
0.0%
260
94.23%
5.77%
1
0.10%
3
0.29%
0
0.0%
238
23.20%
242
98.35%
1.65%
251
98.80%
1.20%
267
91.76%
8.24%
259
94.59%
5.41%
249
95.58%
4.42%
1026
95.13%
4.87%

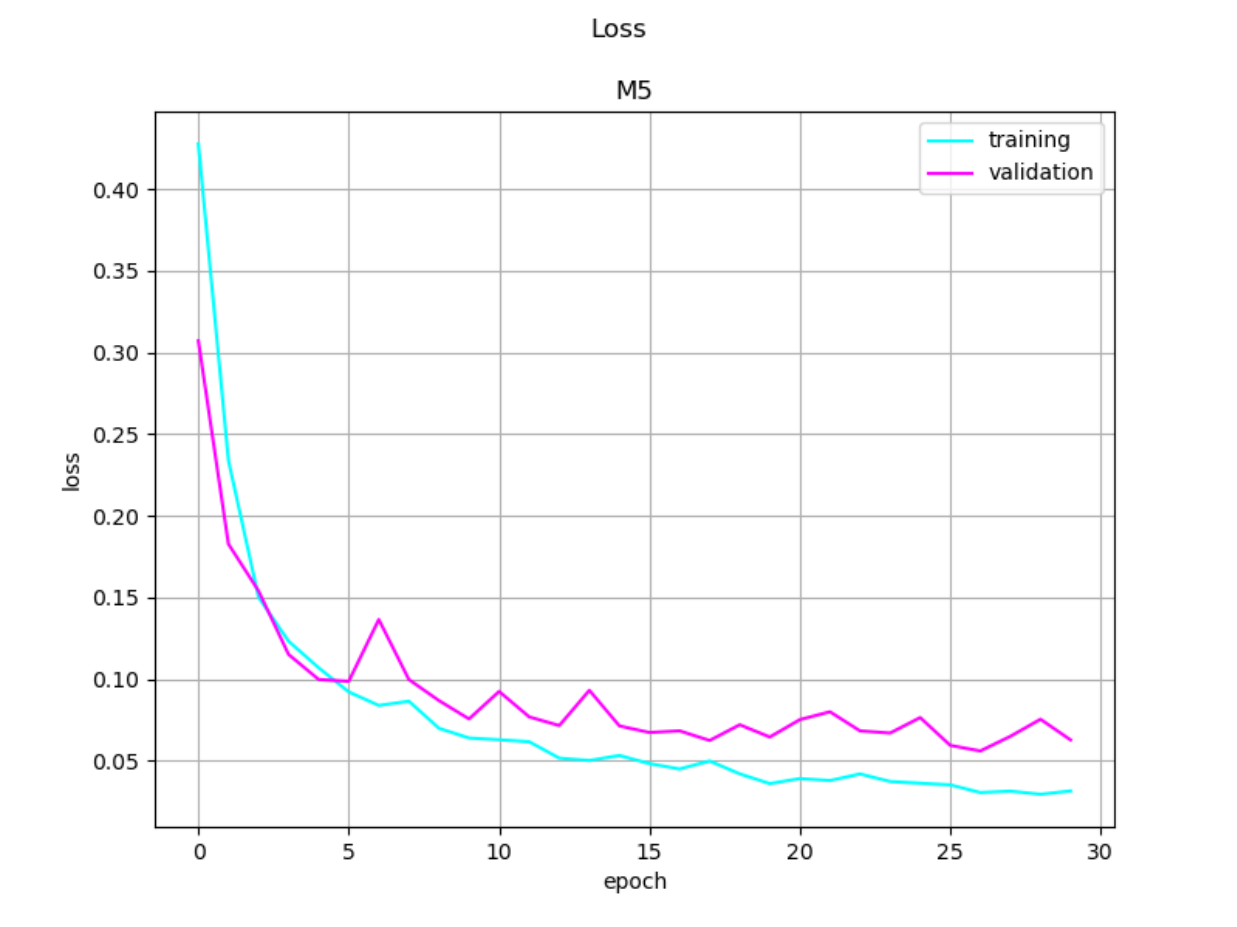
Trénovanie LSTM-CNN na spektrograme

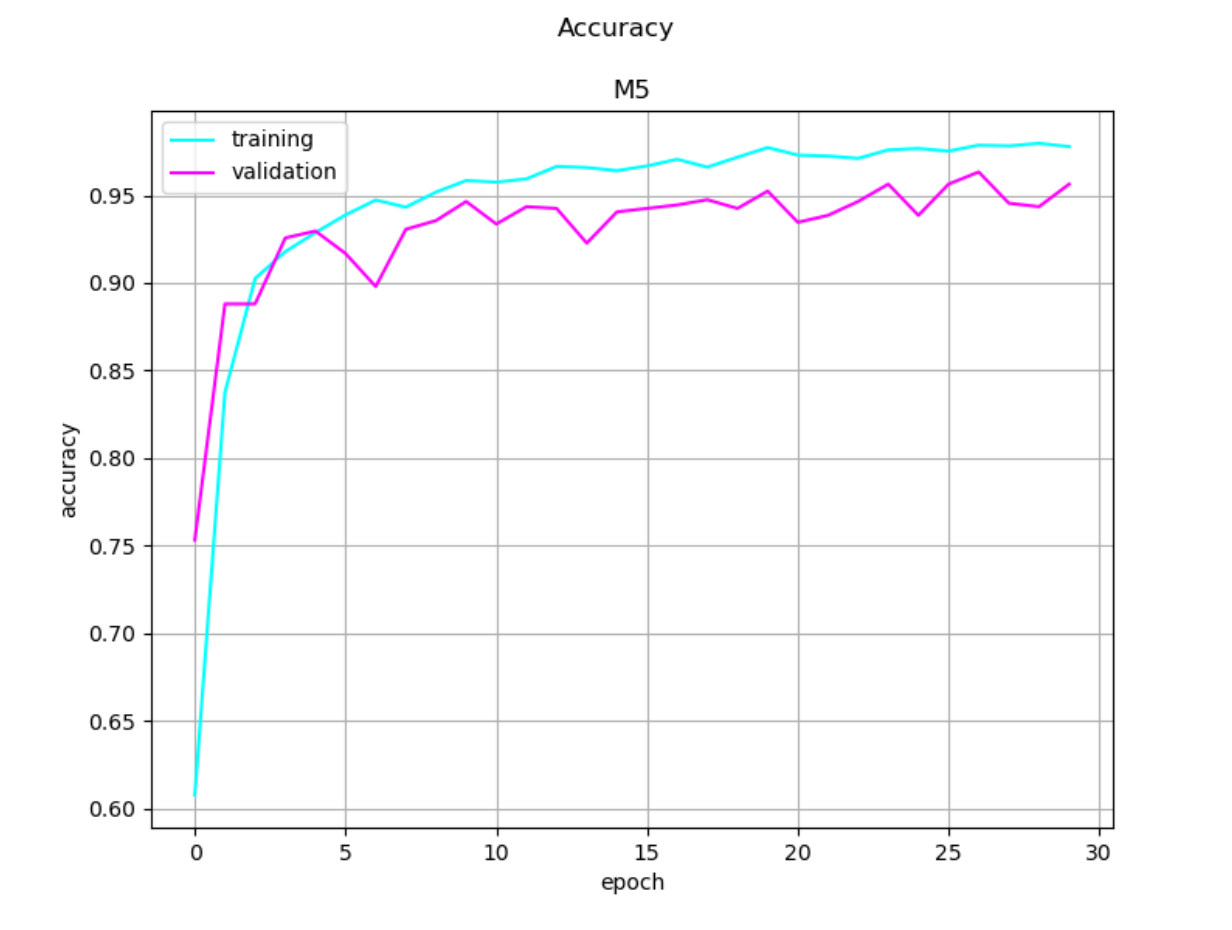


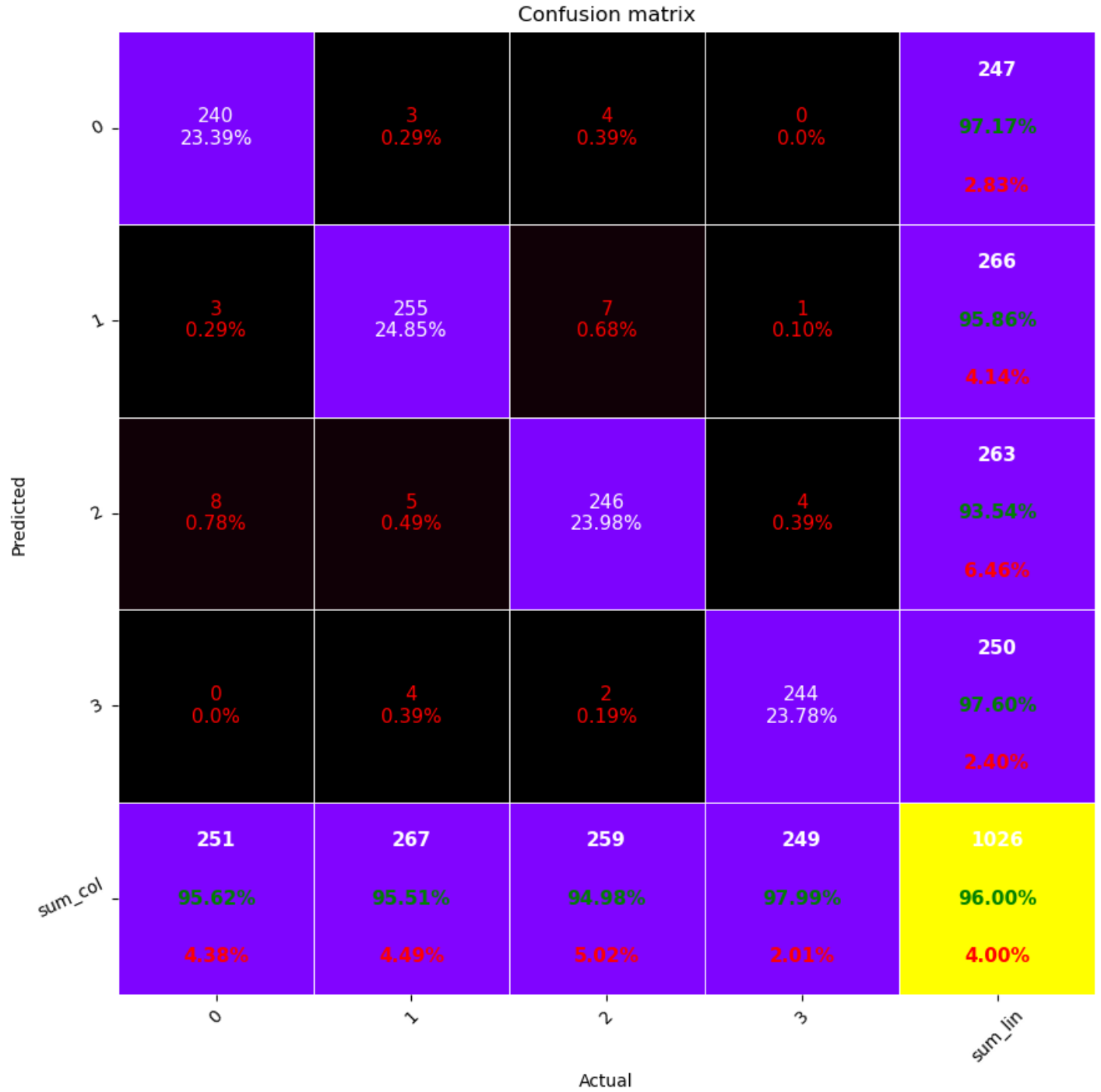



Confusion matrix
Predicted
Actual
0
1
2
3
sum_col
sum_lin
212
20.66%
6
0.58%
3
0.29%
4
0.39%
225
94.22%
5.78%
4
0.39%
241
23.49%
10
0.97%
4
0.39%
259
93.05%
6.95%
23
2.24%
15
1.46%
244
23.78%
4
0.39%
286
85.31%
14.69%
12
1.17%
5
0.49%
2
0.19%
237
23.10%
256
92.58%
7.42%
251
84.46%
15.54%
267
90.26%
9.74%
259
94.21%
5.79%
249
95.18%
4.82%
1026
91.03%
8.97%

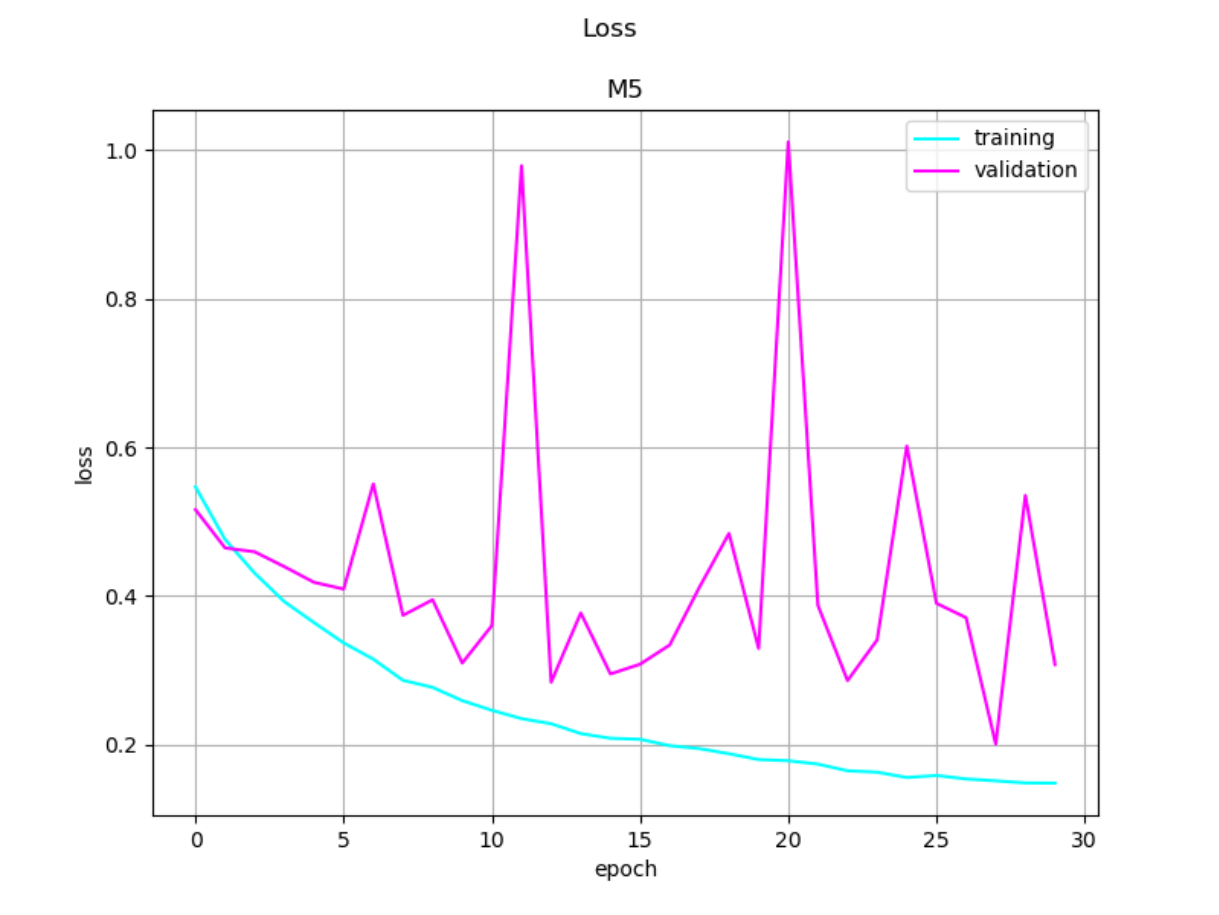
Trénovanie M5 na RAW dátach

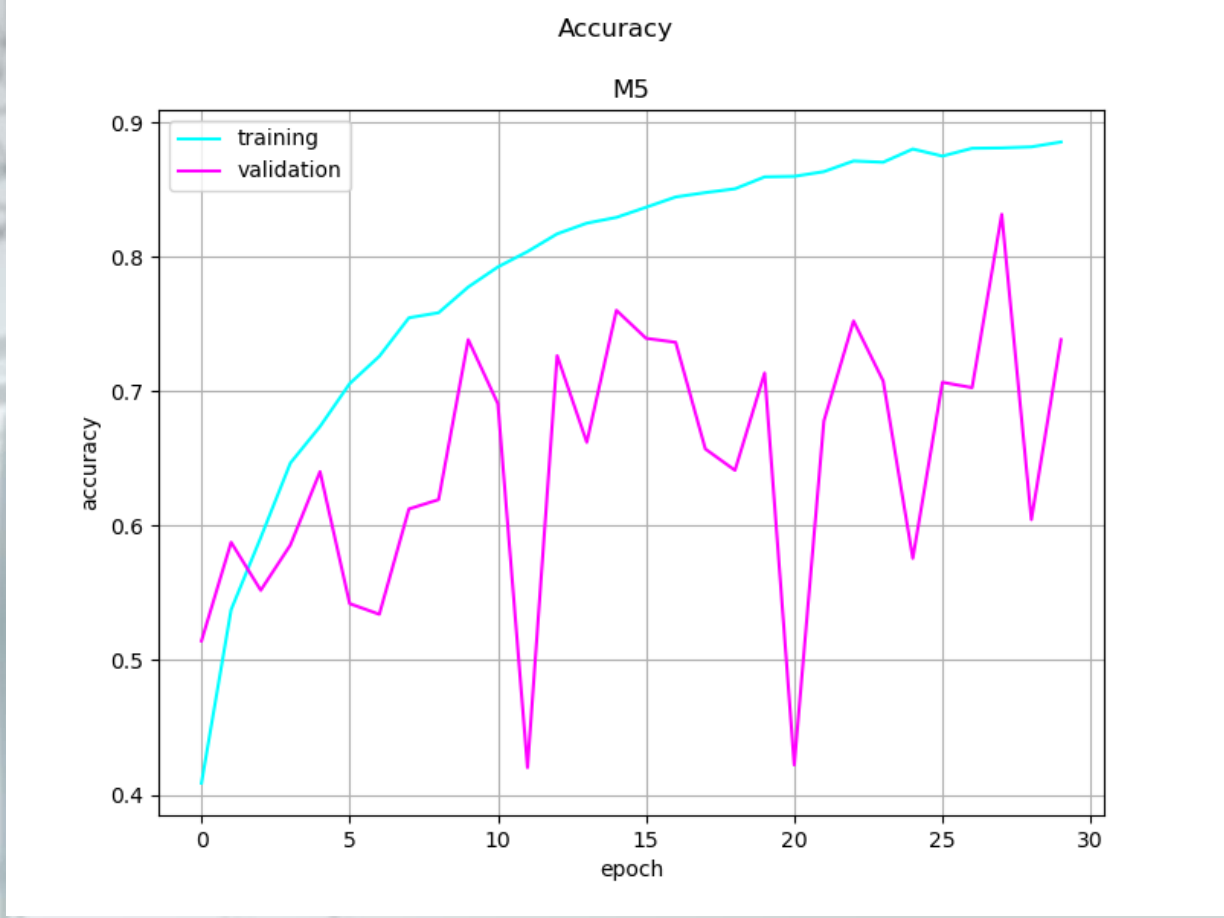


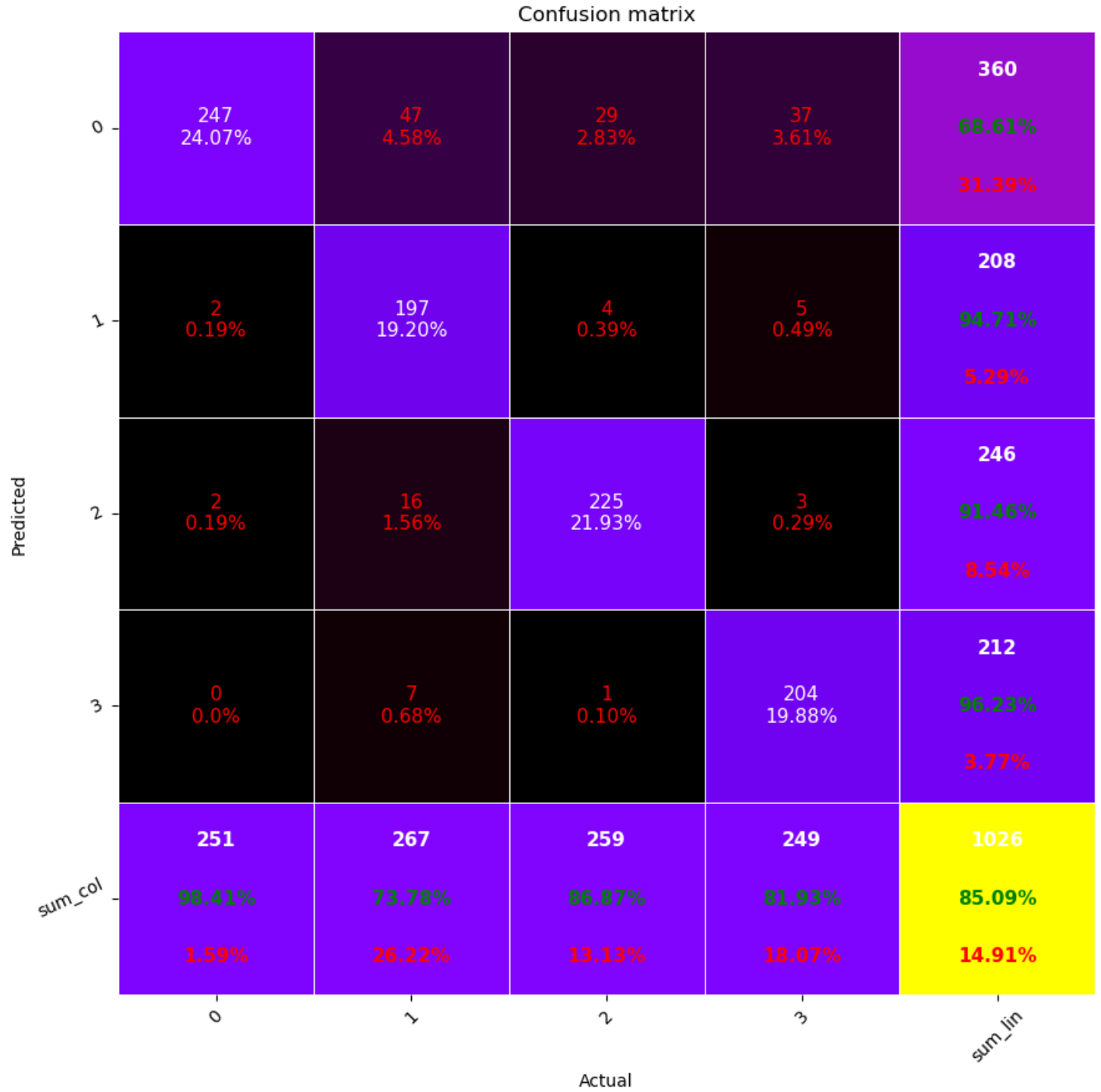



Confusion matrix
Predicted
Actual
0
1
2
3
sum_col
sum_lin
240
23.39%
3
0.29%
4
0.39%
0
0.0%
247
97.17%
2.83%
3
0.29%
255
24.85%
7
0.68%
1
0.10%
266
95.86%
4.14%
8
0.78%
5
0.49%
246
23.98%
4
0.39%
263
93.54%
6.46%
0
0.0%
4
0.39%
2
0.19%
244
23.78%
250
97.60%
2.40%
251
95.62%
4.38%
267
95.51%
4.49%
259
94.98%
5.02%
249
97.99%
2.01%
1026
96.00%
4.00%

Trénovanie M5 na spektrograme






Confusion matrix
247
24.07%
47
4.58%
29
2.83%
37
3.61%
360
68.61%
31.39%
2
0.19%
197
19.20%
4
0.39%
5
0.49%
208
94.71%
5.29%
2
0.19%
16
1.56%
225
21.93%
3
0.29%
246
91.46%
8.54%
0
0.0%
7
0.68%
1
0.10%
204
19.88%
212
96.23%
3.77%
251
98.41%
1.59%
267
73.78%
26.22%
259
86.87%
13.13%
249
81.93%
18.07%
1026
85.09%
14.91%
Predicted
0
1
2
3
sum_col
Actual
0
1
2
3
sum_lin


DAKKJEME
ZA POZORNOSTT